

コンパイラ (12)

2025/11/11

大域的最適化

基本ブロックを越える最適化

ループ最適化

手続き呼び出し最適化

データフロー解析

静的単一代入 (SSA) 解析

ループ最適化 (1/3)

ループ内のコード実行回数: ループ外のコードよりも一般に多い
ループ内のコードは慎重に最適化

本当に毎回実行しなければならないか？できるだけ効率よく実行

ループ不変式の移動

ループから影響を受けない式を外に出す！

例 6.8

ループ内の演算の強さの軽減

強い演算：時間のコストが大きい演算

足し算 < 掛け算 (加算器 < 乗算器)

例 6.9

例

例 6.8

```
i=1;  
do{  
    c=a/b;x=i*c;...i++;  
} while(i<n);
```

 $\Rightarrow$

```
i=1;t=a/b;  
do{  
    c=t;x=i*c;...i++;  
} while(i<n);
```

```
for(i=1;i<n;i++) {  
    c=a/b;x=1*c; ...  
}
```

 $\Rightarrow$

```
if(n>1) {  
    t=a/b;  
    for(i=1;i<n;i++) {  
        c=t;x=i*c;...  
    }  
}
```

例 6.9

```
for(i=1;i<n;i++)  
    a=a+b*i;
```

 $\Rightarrow$

```
t=0;  
for(i=1;i<n;i++) {  
    t=t+b;a=a+t;  
}
```

ループ最適化 (2/3)

帰納変数消去

帰納変数 (ループを制御する変数) は減らす。
参照数の減少

```
t=b;
for(i=1;i<n;i++) {           m=n*b;
    a=a+t;t=t+b;             for(t=b;t<m;t=t+b)
}                             =>    a=a+t;
```

ループ融合とループ分離

ループ展開とソフトウェアパイプライン

ループ展開とソフトウェアパイプライン

```

for(i=1;i<1000;i++) {
   $A_i$ :a[i]=a[i-1]+1;
   $B_i$ :b[i]=a[i]+b[i-1];
   $C_i$ :c[i]=b[i]+2;
}

for(i=3;i<1000;i++) {
   $C_{i-2}$ :c[i]=b[i]+2;
   $B_{i-1}$ :a[i]=a[i-1]+1;
   $A_i$ :b[i]=a[i]+b[i-1];
}

⇒  $C_{998}$ ;  $B_{999}$ ;
    $C_{999}$ ;

```

	P_1	P_2	P_3
1	A_1		
2	B_1	A_2	
3	C_1	B_2	A_3
4	A_4	C_2	B_3
5	B_4	A_5	C_3
⋮		⋮	

手続き呼出し最適化

インライン展開:

手続き呼出し=駆動レコードを用いてスタック上に情報を保存

操作としてはコストが大きい

スタック操作: 実メモリへの正確な順序での書き込みを要求
短い手続きや関数であればそのまま置き換えてしまった方が
効率がよい。

末尾再帰:

手続きの最後の再帰よびだしは、goto に置き換え可能。

呼出しから復帰しても駆動レコード上の局所環境を使って実行する命令が存在しない。局所領域も再利用可能

末尾再帰の例

```
_Bool ans;  
int a[10000];  
void Bsearch(int x,int i,int j){  
    int m;  
    if (i>j) ans=false;  
    else { m=(i+j)/2;  
        if(x>a[m])  Bsearch(x,m+1,j);  
        else if(x<a[m])  Bsearch(x,i,m-1);  
        else ans=true;  
    }  
}
```

 $\Rightarrow$

```
_Bool ans;  
int a[10000];  
void Bsearch(int x,int i,int j){  
    int m;  
L:  
    if (i>j) ans=false;  
    else { m=(i+j)/2;  
        if(x>a[m])  {i=m+1;goto L;}  
        else if(x<a[m])  {j=m-1;goto L;}  
        else ans=true;  
    }  
}
```



```
_Bool ans;  
while(true){  
    if(i>j) {ans=false;break;}  
    else { m=(i+j)/2;  
        if(x>a[m]) i=m+1;  
        else if(x<a[m]) j=m-1;  
        else {ans=true;break;}  
    }  
}
```

データフロー解析

(講義資料 6.4: 詳細については省略)

基本ブロック B に対するデータフロー

$$out[B] = gen[B] \cup (in[B] - kill[B])$$

プログラム上での位置 (行番号):

$in[B]$ B に入力されるデータ

$out[B]$ B から出力されるデータ

$gen[B]$ B で定義されるデータ

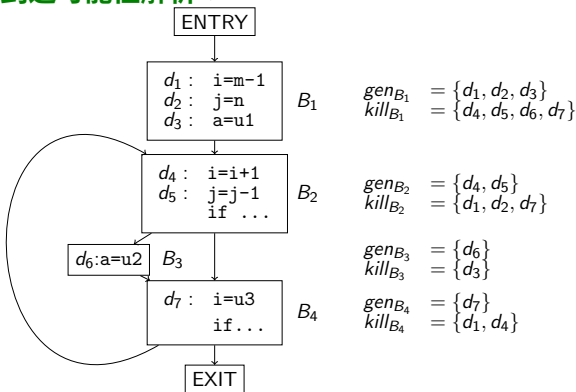
$kill[B]$ B で上書きされるデータ

入力から出力への制御がきまっていればよいので、基本ブロック単位で考えればよい。

ループ: データフロー方程式を解いて全体的に必要なデータを計算する。

データフロー方程式

定義到達可能性解析：



$$IN[B_2] = OUT[B_1] \cup OUT[B_4]$$

$$OUT[B_2] = gen_{B_2} \cup (IN[B_2] - kill_{B_2})$$

(Aho ら：Compilers より引用)

データフロー方程式

$$\begin{aligned} OUT[B_1] &= gen_{B_1} \\ IN[B_2] &= OUT[B_1] \cup OUT[B_4] \\ OUT[B_2] &= gen_{B_2} \cup (IN[B_2] - kill_{B_2}) \\ IN[B_3] &= OUT[B_2] \\ OUT[B_3] &= gen_{B_3} \cup (IN[B_3] - kill_{B_3}) \\ IN[B_4] &= OUT[B_3] \cup OUT[B_2] \\ OUT[B_4] &= gen_{B_4} \cup (IN[B_4] - kill_{B_4}) \end{aligned}$$

データフロー方程式の計算

到達可能性解析：

XXXX XXX: $d_1, d_2, \dots, d_7$ が集合に入っているとき 1、入っていないとき 0

Block B	$OUT[B]^0$	$IN[B]^1$	$OUT[B]^1$	$IN[B]^2$	$OUT[B]^2$
B_1	111 0000	000 0000	111 0000	000 0000	111 0000
B_2	000 1100	111 0001	001 1100	111 0111	001 1110
B_3	000 0010	000 1100	000 1110	001 1100	000 1110
B_4	000 0001	000 1110	000 0111	001 1110	001 0111

変化しなくなるまでくりかえす。

$OUT[B_4]=001\ 0111$:

本ブロックでは d_3, d_5, d_6, d_7 が EXIT が到達する可能性がある。

データフロー

生存解析

$Use[B]$: B で再定義される前に資料される変数の集合

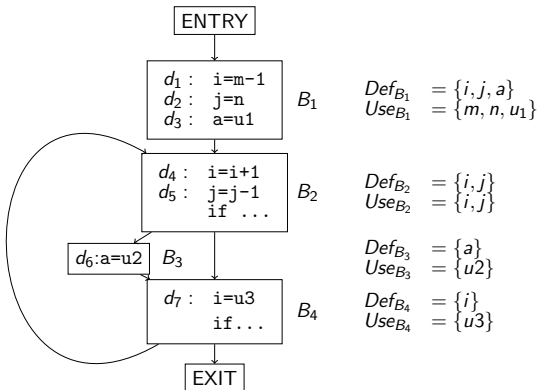
$Def[B]$: B で定義される変数の集合

$$In[B] = Use[B] \cup (Out[B] - Def[B])$$

$$Out[B] = \bigcup_{S \in Succ[B]} In[S]$$

$Succ[B]$: フローグラフにおいて B から直接フローがあるブロック

データフロー解析



データフロー解析（生存解析）

 $(a, i, j, m, n, u1, u2, u3)$ とすると

	OUT	IN	OUT	IN	OUT	IN
B_1	(00000000)	(00011100)	(01100000)	(00011100)	(01100010)	(00011100)
B_2	(00000000)	(01100000)	(01100010)	(01100010)	(01100001)	(01100010)
B_3	(00000000)	(00000010)	(00000001)	(00000010)	(00100001)	(10100001)
B_4	(00000000)	(00000001)	(01100000)	(00100001)	(01100010)	(00100001)

フローグラフの基本ブロックにおいてどの変数が影響を受けるかを解析

	IN	OUT
B_1	$\{m, n, u1\}$	$\{i, j, u2\}$
B_2	$\{i, j, u2\}$	$\{i, j, u3\}$
B_3	$\{a, j, u3\}$	$\{j, u3\}$
B_4	$\{j, u3\}$	$\{i, j, u2\}$

静的単一代入を用いた最適化

データフロー解析のもうひとつの手法: 静的単一代入 (Static Single Assignment: SSA)

各変数の定義がプログラム中で唯一

変数の値は変わらないとする。二度目の代入が発生すると変数名を変更

値が同じの共通部分式が判別できる=式として一致

変換例: 例 6.20、6.21

定義ごとに変数に添字をつける。合流する場合は、 ϕ 関数で表現
いかにして、SSA 形式を得るか?

- (1) ϕ 関数の挿入
- (2) 変数の添字の決定

SSA 形式において最適化

- (3) SSA から逆変換する。(ϕ 関数の消去)

近年のコンパイラでよく用いられる手法

例

```
scanf("%d",&x);
a=1;
b=2;
c=a+b;
if(x>0) a=3;
d=b;
c=a+b;
```

⇒

```
t=1
n=10
l1: t=t*n
n=n-1
b=n>1
if b goto l1
f=t
```

⇒

```
scanf("%d",&x0);
a0=1;
b0=2;
c0=a0+b0;
if(x>0) a1=3;
a2=φ(a0,a1);
d0=b0;
c1=a2+b0;
```

```

t1=1
n1=10
l1: t2=φ(t1,t3)
n2=φ(n1,n3)
t3=t2*n2
n3=n2-1
b1=n3>1
if b1 goto l1
f1=t3
```

ϕ 関数の挿入

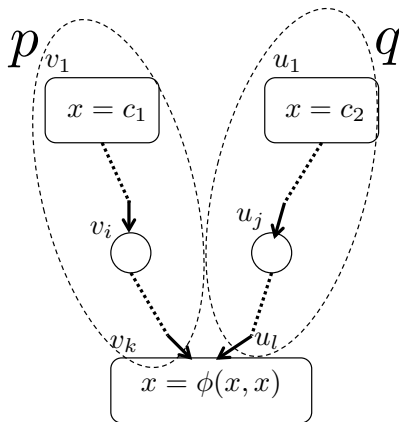
根つきグラフ： (V, E) 先行頂点を持たない唯一の接点 v_o (根) が存在

定義 6.22 定義合流条件：変数 x の ϕ 関数が頂点 v に挿入されるための必要十分条件は、つぎの条件 (1)~(4) を満たす二つの路 $p (= v_1 \dots v_k)$, $q (= u_1 \dots u_l)$ が存在することである。

- (1) p と q は共に長さ 1 以上
- (2) v_1 と u_1 の両方に x の定義が存在する
- (3) $v_k = u_l = v$
- (4) $\{v_1, \dots, v_{k-1}\} \cap \{u_1, \dots, u_{l-1}\} = \emptyset$

パス p 上の x の定義 $=$ **パス q 上の x の定義** $=$ x_2 **定義合流条件の v に、** $x_3 = \phi(x_1, x_2)$

定義合流条件



支配関係

支配関係：開始頂点からのパスに必ず含まれるとき、「支配」される。

定義 6.23 フローグラフを $G = (V, E)$ とする。頂点 $u, v \in V$ に対して、開始頂点 v_0 から v へ至るすべての路 p において u が p 上にある（すなわち、 $p = v_0 v_1 \dots v_k$ かつ $v_k = v$ とすると、 $u = v_i$ を満たす $i \in \{0, \dots, k\}$ が存在する）とき、 u は v を支配する (dominate) という。この支配するという関係を $\geq$ で示す。定義より、この関係は反射律（すべての $u \in V$ に対して $u \geq u$ ）および推移律（ $u \geq v$ かつ $v \geq w$ ならば $u \geq w$ ）を満たす。 $u \geq v$ かつ $u \neq v$ のとき、 $u \gg v$ と書く。このとき、 u は v を強く支配する (strictly dominate) という。 $u \geq v$ のとき、 u を v の支配頂点という。 v の支配頂点の集合を $\text{Dom}(v)$ とする。任意の頂点 $v \in V$ に対して $v_0 \in \text{Dom}(v)$ より、 $\text{Dom}(v) \neq \emptyset$ が成立する。 $\text{Dom}(v) \setminus \{v\}$ 中の最小要素を $\text{idom}(v)$ と書く（すなわち、任意の $u \in \text{Dom}(v) \setminus \{v\}$ に対して、 $u \geq \text{idom}(v) \gg v$ が成立する）。 v が開始頂点でないとき、 $\text{Dom}(v) \setminus \{v\}$ は空でなく、 $\text{idom}(v)$ はただ一つ存在する（この証明は容易である）。

$u \geq v$: u が v を支配: 支配関係の表現=支配木 (図 6.7)

支配関係

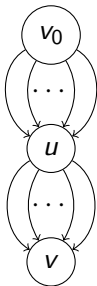
支配関係：開始頂点からのパスに必ず含まれるとき、「支配」される。

$$u \ggg v$$

$$u \ggg u$$

$$u \ggg v, v \ggg w \text{ ならば } u \ggg w$$

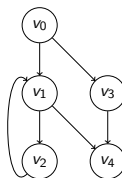
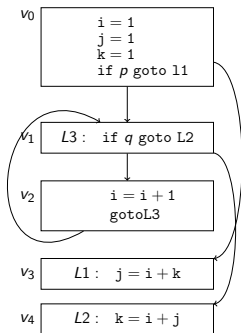
$$u \neq v \text{ のとき } u \ggg v$$



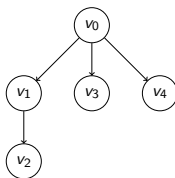
v 以外で最も下にある支配頂点 u : $idom(v)$

$(idom(v), v)$ を辺とするグラフ = **支配木**

フローグラフ → 支配木



フローグラフ



支配木